Basics of Applied Numerical Analysis – HW2

Anastasia Golubkova

10, October 2026

The code for the problems (which is also typed here) is available here: https://github.com/Nastya-golubkova/BoANA/blob/main/hw2/hw2.ipynb

Problem 1. In class we discussed both the clamped and natural spline interpolants through N data points. The clamped spline is determined by prescribing the derivatives at the endpoints, while the natural spline is determined by prescribing zero, or otherwise fixed, curvatures (second derivatives) at the endpoints. For example, the clamped spline interpolant of the data $(x_j, y_j)_{j=1}^N$ is determined by the equations

$$C_1'(x_1) = \alpha, \quad C_{N-1}'(x_N) = \beta, \quad C_i''(x_{i+1}) = C_{i+1}''(x_{i+1}), \quad i = 1, \ldots, N-2. \tag{1}$$

where

$$\begin{aligned} C_i(x) = y_i &+ y[x_i, x_{i+1}](x - x_i) \\ &+ \frac{y[x_i, x_{i+1}] - s_i}{\Delta x_i}(x - x_i)(x - x_{i+1}) \\ &+ \frac{s_{i+1} + s_i - 2y[x_i, x_{i+1}]}{\Delta x_i^2}(x - x_i)^2(x - x_{i+1}) \end{aligned} \tag{2}$$

The unknowns in the system (1) are the components of the vector s, and more explicitly this tridiagonal system is $s_1 =, s_N =,$ and for i = 1, . . . , N - 2

$$\Delta x_{i+1} s_i + 2(\Delta x_i + \Delta x_{i+1}) s_{i+1} + \Delta x_i s_{i+1} = 3(\Delta x_{i+1} y[x_i, x_{i+1}] + \Delta x_i y[x_{i+1}, x_{i+2}]) \tag{3}$$

Write a function (your preferred programming language, this is just a sample)

function s=GetSplineSlopes(x,y,alpha,beta,type)

that returns the slopes s which determine the clamped/natural spline, based on type = C/N, through the data x,y with first/second derivative information alpha,beta at the left and right endpoints respectively.

Solution.

```python
import numpy as np

def GetSplineSlopes(x, y, alpha, beta, type="C"):

    x = np.asarray(x, dtype=float)
    y = np.asarray(y, dtype=float)
    dx = np.diff(x)
    ydiv = np.diff(y) / dx
    N = len(x)

    A = np.zeros((N, N))
    rhs = np.zeros(N)

    for i in range(N - 2):
        A[i + 1, i] = dx[i + 1]
        A[i + 1, i + 1] = 2 * (dx[i] + dx[i + 1])
        A[i + 1, i + 2] = dx[i]
        rhs[i + 1] = 3 * (dx[i + 1] * ydiv[i] + dx[i] * ydiv[i + 1])

    if type == "C":
        A[0, 0] = 1.0
        rhs[0] = alpha
        A[-1, -1] = 1.0
```

```
        rhs[-1] = beta
    if type == "N":
        A[0, 0] = 2.0
        A[0, 1] = 1.0
        rhs[0] = 3 * ydiv[0] - 0.5 * alpha * dx[0]
        A[-1, -2] = 1.0
        A[-1, -1] = 2.0
        rhs[-1] = 3 * ydiv[-1] + 0.5 * beta * dx[-1]


    slopes = np.linalg.solve(A, rhs)
    return slopes
```

Problem 2. Use GetSplineSlopes from Problem 1 (with necessary modifications) and write a function for plotting Hermite's spline to compute and plot the following splines through the for $f(x) = e^{-2x}\sin(10\pi x)$ on an interval $x \in [0, 1]$ with n evenly spaced points:

1. The natural spline for n = 9 and n = 17 (two separate plots). In each plot show the spline interpolant (solid line), the data (e.g. stars or crosses), and the function y = f(x) (dashed line).
2. Repeat, using the clamped spline with the correct end conditions
$$\alpha = f'(0), \beta = f'(1).$$
3. Repeat, using the clamped spline with the the end conditions
$$\alpha = -1, \beta = -1.$$

Solution.

```
import matplotlib.pyplot as plt

def f(x):
    return np.exp(-2 * x) * np.sin(10 * np.pi * x)

def fprime(x):
    return np.exp(-2 * x) * (-2 * np.sin(10 * np.pi * x) + 10 * np.pi * np.cos
        (10 * np.pi * x))


def eval_hermite_spline(x_nodes, y, slopes, x_eval):
    x_nodes = np.asarray(x_nodes, float)
    y = np.asarray(y, float)
    slopes = np.asarray(slopes, float)
    x_eval = np.asarray(x_eval, float)
    S = np.empty_like(x_eval)
    for k, xk in enumerate(x_eval):
        i = int(np.clip(np.searchsorted(x_nodes, xk, side="right") - 1, 0, len(
            x_nodes) - 2))
        h = x_nodes[i + 1] - x_nodes[i]
        t = (xk - x_nodes[i]) / h
        S[k] = (
            y[i] * (2 * t**3 - 3 * t**2 + 1)
            + h * slopes[i] * (t**3 - 2 * t**2 + t)
            + y[i + 1] * (-2 * t**3 + 3 * t**2)
            + h * slopes[i + 1] * (t**3 - t**2)
        )
    return S


def plot_hermite_spline(n, kind, alpha=0.0, beta=0.0, title=None):
    x = np.linspace(0, 1, n)
    y = f(x)
```

```
    s = GetSplineSlopes(x, y, alpha, beta, type=kind)

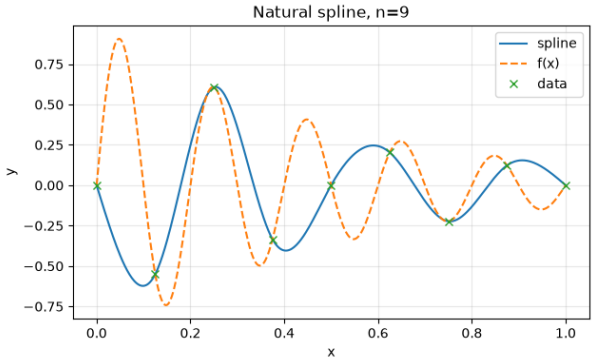
    xx = np.linspace(0, 1, 400)
    S = eval_hermite_spline(x, y, s, xx)

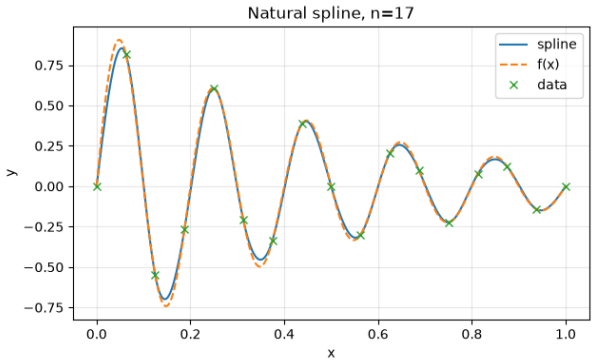
    plt.figure(figsize=(7, 4))
    plt.plot(xx, S, "-", label="spline")
    plt.plot(xx, f(xx), "--", label="f(x)")
    plt.plot(x, y, "x", label="data")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title(title or f"{kind}, n={n}")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()
    return s

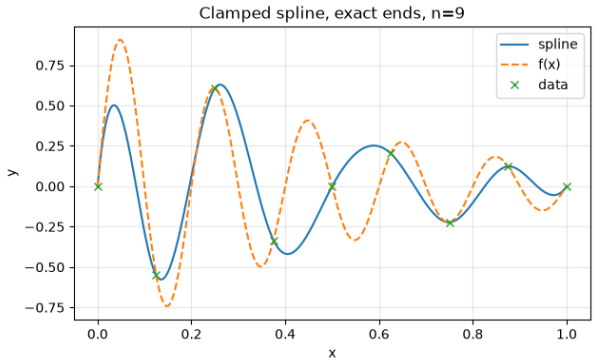

plot_hermite_spline(9, "N", 0, 0, "Natural spline, n=9")
plot_hermite_spline(17, "N", 0, 0, "Natural spline, n=17")

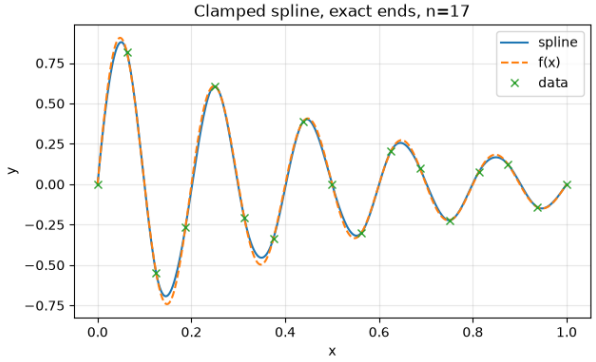
plot_hermite_spline(9, "C", fprime(0), fprime(1), "Clamped spline, exact ends, n=9")
plot_hermite_spline(17, "C", fprime(0), fprime(1), "Clamped spline, exact ends, n=17")

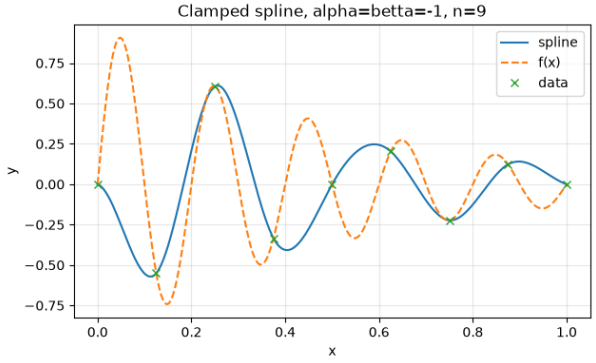
plot_hermite_spline(9, "C", -1, -1, "Clamped spline, alpha=betta=-1, n=9")
plot_hermite_spline(17, "C", -1, -1, "Clamped spline,   =  =-1, n=17")
```

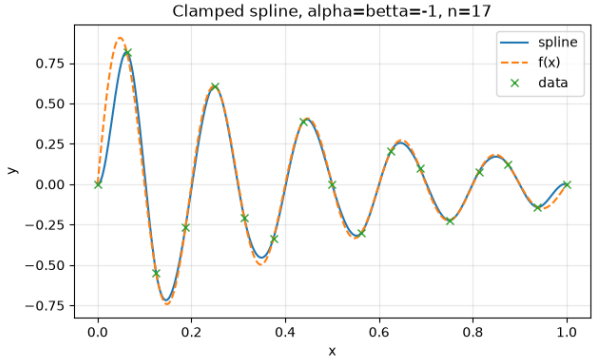












Problem 3. Derive the equations for the coefficients of the clamped spline given in Problem 1. Credit will only be given for a clear and complete derivation.

Solution.

Suppose we are given N data points

$$(x_1, y_1), \ldots, (x_N, y_N), \qquad x_1 < x_2 < \cdots < x_N.$$

On each interval $[x_i, x_{i+1}]$, we define a cubic polynomial

$$C_i(x) = y_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3,$$

where $i = 1, \ldots, N-1$.
The coefficients b_i, c_i, d_i are unknown. The constant term is y_i, so the condition

$$C_i(x_i) = y_i$$

is automatically satisfied.
The interpolation condition at the right endpoint of each interval gives

$$\begin{aligned} y_2 &= C_1(x_2) \\ &= y_1 + b_1(x_2 - x_1) + c_1(x_2 - x_1)^2 + d_1(x_2 - x_1)^3, \end{aligned}$$

and, in general,

$$\begin{aligned} y_{i+1} &= C_i(x_{i+1}) \\ &= y_i + b_i(x_{i+1} - x_i) + c_i(x_{i+1} - x_i)^2 \\ &\quad + d_i(x_{i+1} - x_i)^3, \qquad i = 1, \ldots, N-1. \end{aligned}$$

Thus, interpolation gives $N-1$ equations for the coefficients.
A cubic spline must have a continuous first derivative at each interior knot:

$$C_i'(x_{i+1}) = C_{i+1}'(x_{i+1}), \qquad i = 1, \ldots, N-2.$$

Differentiating the polynomial gives

$$C_i'(x) = b_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2.$$

Since

$$C_{i+1}'(x_{i+1}) = b_{i+1} + 2c_{i+1}(x_{i+1} - x_{i+1}) + 3d_{i+1}(x_{i+1} - x_{i+1})^2 = b_{i+1},$$

the continuity condition becomes

$$b_i + 2c_i(x_{i+1} - x_i) + 3d_i(x_{i+1} - x_i)^2 = b_{i+1}.$$

There are $N-2$ such equations.
The second derivative must also be continuous at each interior knot:

$$C_i''(x_{i+1}) = C_{i+1}''(x_{i+1}), \qquad i = 1, \ldots, N-2.$$

Differentiating again gives

$$C_i''(x) = 2c_i + 6d_i(x - x_i).$$

Therefore,

$$C_i''(x_{i+1}) = 2c_i + 6d_i(x_{i+1} - x_i),$$

while

$$C_{i+1}''(x_{i+1}) = 2c_{i+1}.$$

Equating these expressions yields

$$2c_i + 6d_i(x_{i+1} - x_i) = 2c_{i+1}.$$

Again, there are $N-2$ equations.
For a clamped cubic spline, the first derivatives at both endpoints are prescribed:

$$C_1'(x_1) = \alpha, \qquad C_{N-1}'(x_N) = \beta.$$

Using the formula for the first derivative, we obtain

$$b_1 = \alpha$$

and

$$C'_{N-1}(x_N) = \beta = b_{N-1} + 2c_{N-1}(x_N - x_{N-1}) + 3d_{N-1}(x_N - x_{N-1})^2.$$

These are the two additional boundary equations.
There are $3(N-1)$ unknown coefficients:

$$b_1, \ldots, b_{N-1}, \quad c_1, \ldots, c_{N-1}, \quad d_1, \ldots, d_{N-1}.$$

We have obtained:
- $N-1$ interpolation equations; - $N-2$ first-derivative continuity equations; - $N-2$ second-derivative continuity equations; - 2 clamped boundary equations.
The total number of equations is $(N-1) + (N-2) + (N-2) + 2 = 3N - 3$.
This equals the number of unknown coefficients, so we obtain a square linear system that determines the clamped cubic spline.
We begin with a general cubic polynomial on the (i)-th interval:

$$C_i(x) = y_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3.$$

The constant term is (y_i) because the interpolation condition requires $C_i(x_i) = y_i$.
We now introduce the notation

$$s_i = C'(x_i), \qquad i = 1, \ldots, N,$$

where s_i is the derivative of the spline at the (i)-th data point. At an interior knot x_i, the spline is required to have a continuous first derivative, so the derivatives of the two adjacent segments agree.
Therefore, s_i is a single well-defined slope at each knot.
For the i-th segment, differentiating the polynomial gives

$$C'_i(x) = b_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2.$$

Evaluating at x_i, we obtain

$$C'_i(x_i) = b_i = s_i.$$

Similarly, the derivative at the right endpoint of the segment must be s_{i+1}. Thus, instead of working directly with the three coefficients (b_i, c_i, d_i) on every interval, we can express the polynomial using the slopes s_i and s_{i+1}.
Define the interval length and the secant slope by

$$h_i = x_{i+1} - x_i, \qquad m_i = \frac{y_{i+1} - y_i}{h_i}.$$

Since $C_i(x)$ is cubic, it is uniquely determined by the four conditions

$$C_i(x_i) = y_i, \quad C_i(x_{i+1}) = y_{i+1}, \; C'_i(x_i) \quad = s_i, C'_i(x_{i+1}) \quad = s_{i+1}.$$

The first two conditions enforce interpolation, while the last two specify the slopes at the endpoints.
These four conditions determine the four degrees of freedom of a cubic polynomial.
To construct a polynomial satisfying these conditions, write

$$C_i(x) = y_i + s_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3.$$

The condition $C_i(x_{i+1}) = y_{i+1}$ gives

$$s_i h_i + c_i h_i^2 + d_i h_i^3 = y_{i+1} - y_i = m_i h_i,$$

or, after dividing by h_i,

$$c_i h_i + d_i h_i^2 = m_i - s_i.$$

The derivative is

$$C_i'(x) = s_i + 2c_i(x - x_i) + 3d_i(x - x_i)^2.$$

Applying $C_i'(x_{i+1}) = s_{i+1}$ gives

$$2c_i h_i + 3d_i h_i^2 = s_{i+1} - s_i.$$

Solving these two equations for c_i and d_i, we obtain

$$c_i = \frac{3m_i - 2s_i - s_{i+1}}{h_i}, \qquad d_i = \frac{s_i + s_{i+1} - 2m_i}{h_i^2}.$$

Consequently, the segment can be written as

$$C_i(x) = y_i + s_i(x - x_i) \ + \frac{3m_i - 2s_i - s_{i+1}}{h_i}(x - x_i)^2 \ + \frac{s_i + s_{i+1} - 2m_i}{h_i^2}(x - x_i)^3.$$

Equivalently, rearranging the polynomial gives the representation

$$C_i(x) = y_i + m_i(x - x_i) \ + \frac{m_i - s_i}{h_i}(x - x_i)(x - x_{i+1}) \ + \frac{s_i + s_{i+1} - 2m_i}{h_i^2}(x - x_i)^2(x - x_{i+1}).$$

The clamped spline prescribes the derivatives at the two endpoints:

$$C_1'(x_1) = \alpha, \qquad C_{N-1}'(x_N) = \beta.$$

By the definition of the knot slopes, these conditions become

$$s_1 = \alpha, \qquad s_N = \beta.$$

For the spline to be twice continuously differentiable, the second derivatives of adjacent segments must agree at every interior knot:

$$C_i''(x_{i+1}) = C_{i+1}''(x_{i+1}), \qquad i = 1, \ldots, N - 2.$$

Differentiating the cubic representation twice and evaluating at the shared knot gives

$$C_i''(x_{i+1}) = \frac{2}{h_i}(s_i + 2s_{i+1} - 3m_i)$$

and

$$C_{i+1}''(x_{i+1}) = \frac{2}{h_{i+1}}(3m_{i+1} - 2s_{i+1} - s_{i+2}).$$

Equating these expressions and multiplying by $h_i h_{i+1}/2$
yields

$$h_{i+1}(s_i + 2s_{i+1} - 3m_i) = h_i(3m_{i+1} - 2s_{i+1} - s_{i+2}).$$

Collecting the slope terms on the left and the secant-slope terms on the right gives

$$h_{i+1}s_i + 2(h_i + h_{i+1})s_{i+1} + h_i s_{i+2} \ = 3(h_{i+1}m_i + h_i m_{i+1}),\ for (i = 1, \ldots, N - 2).$$

The equations above, together with

$$s_1 = \alpha, \qquad s_N = \beta,$$

form a tridiagonal linear system for the N unknown knot slopes $s_1, \ldots, s_N$. Once this system is solved, the slopes can be substituted into the cubic segment formula to determine every spline polynomial $C_i(x)$.

Therefore, the spline is constructed in two stages: first, solve for the slopes at the knots; second, use these slopes and the interpolation data to recover the cubic coefficients on each interval.

Problem 4. Suppose you are given a set of data points (x_j, y_j) in the plane that describe a curve that is not a function of x, and we want to interpolate the data. The basic idea is to think of the curve as parametrized by arclength $\lambda : (x(\lambda), y(\lambda))$. Here we use λ to denote arclength in lieu of the typical s, since s was earlier used for slope (derivative) information. Thus the discrete data correspond to discrete values of arclength λ_j . That is, $x_j = x(\lambda_j)$ and $y_j = y(\lambda_j)$. To determine λ_j we use the arclength of the piecewise linear interpolant. For example, the data $(x_1, y_1) = (1, 1) \quad (x_2, y_2) = (2, 2) \quad (x_3, y_3) = (1, 4)$ is viewed as corresponding to arclength values $\lambda_1 = 0, \lambda_2 = \sqrt{2}, \lambda_3 = \sqrt{2} + \sqrt{5}$. Once we have determined the λ_j , we can fit a cubic spline through $(\lambda_j, x_j)_{j=1}^N$ and a second cubic spline through $(\lambda_j, y_j)_{j=1}^N$, and then plot the resulting curve. An interesting example of the technique is the need to approximate curves for the automatic control of sewing machines. Consider the following data.

x=[4.7 4.0 4.6 2.3 2.3 4.7 4.1 8.3 6.0 8.6 7.8 5.4 5.4 4.7];

y=[1.3 1.9 2.5 3.1 3.9 3.1 5.8 5.8 3.0 2.9 2.0 2.4 1.5 1.3];

Fit the data (as a function of arclength) with two natural cubic splines and then plot $(x(\lambda), y(\lambda))$ as well as the data points.

Solution.

```python
import numpy as np
import matplotlib.pyplot as plt

x = np.array([4.7, 4.0, 4.6, 2.3, 2.3, 4.7, 4.1,
              8.3, 6.0, 8.6, 7.8, 5.4, 5.4, 4.7])

y = np.array([1.3, 1.9, 2.5, 3.1, 3.9, 3.1, 5.8,
              5.8, 3.0, 2.9, 2.0, 2.4, 1.5, 1.3])

dx = np.diff(x)
dy = np.diff(y)

dlambda = np.sqrt(dx**2 + dy**2)
lam = np.concatenate(([0], np.cumsum(dlambda)))

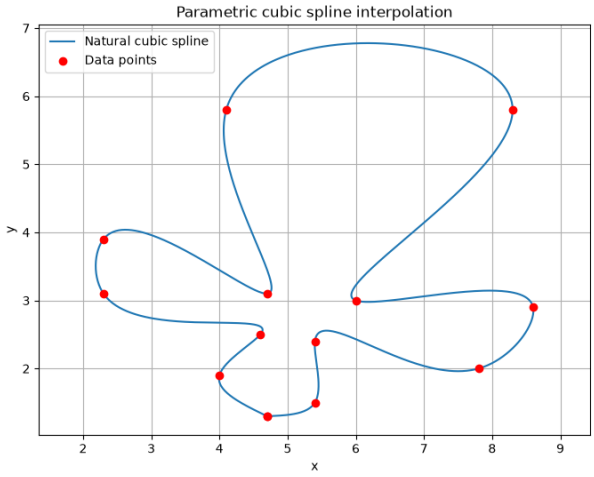


sx_slopes = GetSplineSlopes(lam, x, 0.0, 0.0, type="N")
sy_slopes = GetSplineSlopes(lam, y, 0.0, 0.0, type="N")

lam_fine = np.linspace(lam[0], lam[-1], 1000)
x_fine = eval_hermite_spline(lam, x, sx_slopes, lam_fine)
y_fine = eval_hermite_spline(lam, y, sy_slopes, lam_fine)

plt.figure(figsize=(8, 6))
plt.plot(x_fine, y_fine, label="Natural cubic spline")
plt.scatter(x, y, color="red", zorder=3, label="Data points")

plt.xlabel("x")
plt.ylabel("y")
plt.title("Parametric cubic spline interpolation")
plt.axis("equal")
plt.grid(True)
plt.legend()
plt.show()
```



Problem 5. Computer problem 8, Section 4.1, page 200 of Sauer's textbook. (Hint: Hilbert matrix is given by $H_{ij} = 1/(i + j1)$ for $i, j = 1, 2, 3, ...$)

Let A be the 10 × n matrix formed by the first n columns of the 10 × 10 Hilbert matrix. Let c be the n-vector [1,...,1], and set b = Ac. Use the normal equations to solve the least squares problem Ax = b for (a) n = 6 (b) n = 8, and compare with the correct least squares solution x = c. How many correct decimal places can be computed? Use condition number to explain the results. (This least squares problem is revisited in Computer Problem 4.3.7.)

Solution.

```
1
2  def get_A(n):
3    return np.array([[1/(i + j - 1) for j in range(1, n + 1)] for i in range(1,
         11)])
4
5  def get_c(n):
6    return np.ones(n)
7
8  def get_b(n):
9    return get_A(n) @ get_c(n)
10
11 def least_squares(n):
12   A = get_A(n)
13   b = get_b(n)
14
15   return np.linalg.inv(A.T @ A) @ A.T @ b
16
17
18 print(least_squares(6))
19 print()
20 print(least_squares(8))
21 print()
22 print(f"Condition number of A.T @ A 10*6: {np.linalg.cond(get_A(6).T @ get_A(6)
       )}")
23 print(f"Condition number of A.T @ A 10*8: {np.linalg.cond(get_A(8).T @ get_A(8)
       )}")
24
25
26 output:
27
28 [0.99990819 1.00024825 0.99999254 0.99977193 1.00027157 0.99991855]
29
30 [  6.80081946   -7.89984404 -29.83960048  97.77164881 -73.96655108
31   11.63142238    3.24110382   0.88491136]
32
33 Condition number of A.T @ A 10*6: 6004941862896.767
34 Condition number of A.T @ A 10*8: 5.489063089463325e+17
```

For $n = 6$ the computed solution is $x \approx (0.99991, 1.00025, 0.99999, 0.99977, 1.00027, 0.99992)$, with a maximum componentwise error of about $2.7 \cdot 10^{-4}$. Hence only about **3 decimal places are correct**.
For $n = 8$ the computed solution is garbage (e.g. $x_4 \approx 97.8$ instead of 1, errors up to $\sim 10^2$), so **no digits are correct**.
This is explained by the condition number of the normal equations matrix, $\kappa(A^TA) = \kappa(A)^2$. Solving a linear system in double precision ($\varepsilon_{\text{mach}} \approx 1.1 \cdot 10^{-16}$) loses about $\log_{10} \kappa$ digits out of ~ 16. For $n = 6$, $\kappa(A^TA) \approx 6.0 \cdot 10^{12}$, so about 12–13 digits are lost and only ≈ 3–4 remain, which matches the observed accuracy. For $n = 8$, $\kappa(A^TA) \approx 5.5 \cdot 10^{17} > 1/\varepsilon_{\text{mach}} \approx 10^{16}$, so all significant digits are lost and the computed x has no correct digits.

Problem 6. Find (by hand) a QR factorization of the matrix given in problem 2(b) from Section 4.3, page 224 of Sauer's textbook. Use the Gram-Schmidt procedure described in Sauer's textbook, and compare your answer with the factorization returned by any available function you can find. Matrix is:

$$A = \begin{bmatrix} -4 & -4 \\ -2 & 7 \\ 4 & -5 \end{bmatrix}$$

Solution.
First, we obtain Gramm-Shmidt orthogonalization:

$$y_1 = A_1 = \begin{pmatrix} -4 \\ -2 \\ 4 \end{pmatrix},$$

$$r_{11} = \|y_1\|_2 = \sqrt{(-4)^2 + (-2)^2 + 4^2} = 6,$$

$$q_1 = \frac{y_1}{r_{11}} = \begin{pmatrix} -\frac{2}{3} \\ -\frac{1}{3} \\ \frac{2}{3} \end{pmatrix},$$

$$\begin{aligned} r_{12} = q_1^T A_2 &= \begin{pmatrix} -\frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \end{pmatrix} \begin{pmatrix} -4 \\ 7 \\ -5 \end{pmatrix} \\ &= \frac{8}{3} - \frac{7}{3} - \frac{10}{3} = -3, \end{aligned}$$

$$\begin{aligned} y_2 &= A_2 - r_{12} q_1 \\ &= \begin{pmatrix} -4 \\ 7 \\ -5 \end{pmatrix} - (-3) \begin{pmatrix} -\frac{2}{3} \\ -\frac{1}{3} \\ \frac{2}{3} \end{pmatrix} \\ &= \begin{pmatrix} -6 \\ 6 \\ -3 \end{pmatrix}. \end{aligned}$$

$$r_{22} = ||y_2||_2 = \sqrt{(-6)^2 + 6^2 + (-3)^2} = \sqrt{81} = 9.$$

$$q_2 = \frac{y_2}{r_{22}} = \begin{pmatrix} -\frac{2}{3} \\ \frac{2}{3} \\ -\frac{1}{3} \end{pmatrix}.$$

The QR factorization is

$$Q = \begin{pmatrix} -\frac{2}{3} & -\frac{2}{3} \\ -\frac{1}{3} & \frac{2}{3} \\ \frac{2}{3} & -\frac{1}{3} \end{pmatrix}, \qquad R = \begin{pmatrix} 6 & -3 \\ 0 & 9 \end{pmatrix}.$$

The python function returns:

```
A = np.array([[-4, -2, 4], [-4, 7, -5]])

Q, R = np.linalg.qr(A.T)

print(Q)
print(R)

output:

[[-0.66666667  0.66666667]
 [-0.33333333 -0.66666667]
 [ 0.66666667  0.33333333]]
[[ 6. -3.]
 [ 0. -9.]]
```

The Python function returns the opposite signs for the second column of Q and the second row of R. The same result could be obtained by hand by choosing the opposite sign for q_2 in the Gram–Schmidt procedure and changing the sign of the corresponding diagonal entry r_{22}. Both factorizations are valid, since Q has orthonormal columns, R is upper triangular, and A=QR.

Problem 7. Use your results from Problem 6 to solve (by hand) the least squares problem given in problem 7(b) from Section 4.3, page 224 of Sauer's textbook. Also report the length of the minimum residual.

The system is:

$$\begin{bmatrix} -4 & -4 \\ -2 & 7 \\ 4 & -5 \end{bmatrix} X = \begin{bmatrix} 3 \\ 9 \\ 0 \end{bmatrix}$$

Solution. As were defined in Problem 6:

$$A = \begin{bmatrix} -4 & -4 \\ -2 & 7 \\ 4 & -5 \end{bmatrix} = \begin{pmatrix} -\frac{2}{3} & -\frac{2}{3} \\ -\frac{1}{3} & \frac{2}{3} \\ \frac{2}{3} & -\frac{1}{3} \end{pmatrix} \begin{pmatrix} 6 & -3 \\ 0 & 9 \end{pmatrix}.$$

To solve the least-squares problem, we use the normal equations

$$A^T AX = A^T b.$$

Since $A = QR$, we obtain

$$(QR)^T (QR)X = (QR)^T b.$$

Using $Q^T Q = I$, we get

$$R^T RX = R^T Q^T b.$$

Since R is invertible, this simplifies to

$$RX = Q^T b.$$

For $b = (3, 9, 0)^T$,

$$Q^T b = \begin{pmatrix} -\frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \\ -\frac{2}{3} & \frac{2}{3} & -\frac{1}{3} \end{pmatrix} \begin{pmatrix} 3 \\ 9 \\ 0 \end{pmatrix} = \begin{pmatrix} -5 \\ 4 \end{pmatrix}.$$

Thus,

$$\begin{pmatrix} 6 & -3 \\ 0 & 9 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} -5 \\ 4 \end{pmatrix}.$$

Solving this system gives

$$x_2 = \frac{4}{9}, \qquad 6x_1 - 3 \cdot \frac{4}{9} = -5,$$

so

$$X_{\text{approx}} = \begin{pmatrix} -\frac{11}{18} \\ \frac{4}{9} \end{pmatrix}.$$

The residual is

$$r = b - AX_{\text{approx}} = \begin{pmatrix} \frac{7}{3} \\ \frac{14}{3} \\ \frac{14}{3} \end{pmatrix}.$$

The minimum residual length is

$$\|r\|_2 = \sqrt{\frac{7^2 + 14^2 + 14^2}{9}} = 7.$$

Problem 8. Prove that the closed Newton-Cotes rule $Q_{NC(m)}$ will compute the integral

$$\int_0^1 x^{k-1} dx = \frac{1}{k}$$

exactly for $k = 1, 2, , m$. Here the quadrature nodes are the fixed points $x_j = (j1)/(m1)$ for $j = 1, 2, , m$. Therefore, the weights $w_1, w_2, , w_m$ satisfy

$$\mathrm{w}_1 x_1^{k1} + w_2 x_2^{k1} + \ + w_m x_m^{k1} = \tfrac{1}{k} \ , \ \mathrm{k} = 1, 2, \cdots, \mathrm{m}.$$

These conditions define a linear system. Write a function ComputeClosedNewtonCotesWeights to solve this system for the weights (returning vector of coefficients), and compare your computed weights with those given by M. Abramowitz and I. Stegun or any other source. For comparison you can use a norm of the following defference vector:
|ComputeNewtonCotesClosedWeights(m)-NewtonCotesClosedWeights(m)|, where NewtonCotesClosedWeights(m) vector contains exact weights.

Solution. Proof: The closed Newton-Cotes rule with nodes $x_j = \frac{j-1}{m-1}$, $j = 1, \ldots, m$, is

$$Q_{NC(m)}(f) = \sum_{j=1}^{m} w_j f(x_j).$$

By definition, the weights solve the linear system

$$\sum_{j=1}^{m} w_j x_j^{k-1} = \frac{1}{k}, \qquad k = 1, \ldots, m,$$

that is, $Vw = b$, where $V_{kj} = x_j^{k-1}$ is a Vandermonde matrix and $b_k = 1/k$. Since the nodes are distinct, $\det V = \prod_{i<j}(x_j - x_i) \neq 0$, so the system has a unique solution w. For this w and every $k = 1, \ldots, m$ we have

$$Q_{NC(m)}\big(x^{k-1}\big) = \sum_{j=1}^{m} w_j x_j^{k-1} = \frac{1}{k} = \int_0^1 x^{k-1}\, dx,$$

so the rule is exact for $x^0, \ldots, x^{m-1}$, and hence for all polynomials of degree at most $m - 1$. ■

Below is the function that solves this system and its comparison with the exact weights from Abramowitz–Stegun.

```python

import numpy as np

def ComputeClosedNewtonCotesWeights(m):
    nodes = np.linspace(0, 1, m)

    A = np.vander(nodes, N=m, increasing=True).T
    b = 1 / np.arange(1, m + 1)

    return np.linalg.solve(A, b)

exact_weights = {
    2: np.array([1/2, 1/2]),
    3: np.array([1/6, 4/6, 1/6]),
    4: np.array([1/8, 3/8, 3/8, 1/8]),
    5: np.array([7/90, 32/90, 12/90, 32/90, 7/90]),
    6: np.array([19/288, 75/288, 50/288,
                 50/288, 75/288, 19/288]),
}

for m, exact in exact_weights.items():
    computed = ComputeClosedNewtonCotesWeights(m)
    error = np.linalg.norm(computed - exact)
    print(f"m={m}: error = {error:.3e}")

output:

m=2: error = 0.000e+00
m=3: error = 1.178e-16
m=4: error = 8.729e-16
m=5: error = 1.587e-15
m=6: error = 1.403e-14
```

The computed weights agree with the tabulated ones to about machine precision. The difference is not zero because of rounding errors, and it grows with m ($0 \to 10^{-16} \to 10^{-14}$) since the Vandermonde matrix becomes increasingly ill-conditioned as m increases.

Problem 9. Exercise 5.2.4(c), Sauer's textbook, page 263. Apply the composite Simpson's Rule with m = 1,2, and 4 panels to the integrals, and report the errors.

$$\int_0^{\pi} x \cos x dx$$

Solution. The exact integral is

$$I = \int_0^{\pi} x \cos x \, dx = -2.$$

The composite Simpson's Rule with m panels uses $2m$ subintervals, with $h = \frac{\pi}{2m}$.

$$S_m = \frac{h}{3}\left[f(x_0) + f(x_{2m}) + 4\sum_{i \text{ odd}} f(x_i) + 2\sum_{\substack{i \text{ even} \\ 0<i<2m}} f(x_i)\right].$$

Applying the formula gives:

m	S_m	$\lvert I - S_m \rvert$
1	-1.644934	0.355066
2	-1.985611	0.014389
4	-1.999182	0.000818

The error decreases as the number of panels increases, consistent with the fourth-order accuracy of the composite Simpson's Rule.

Problem 10. The three-point Gauss-Legendre formula for approximating integrals has the form

$$\int_{x_k}^{x_{k+1}} f(x)dx = h[w_1 f(x_k + c_1 h) + w_2 f(x_k + c_2 h) + w_3 f(x_k + c_3 h)], \quad h = x_{k+1} - x_k$$

Consider nonlinear system:

$$\begin{aligned} w_1 + w_2 + w_3 &= 1, \\ w_1 c_1 + w_2 c_2 + w_3 c_3 &= 1/2, \\ w_1 c_1^2 + w_2 c_2^2 + w_3 c_3^2 &= 1/3, \\ w_1 c_1^3 + w_2 c_2^3 + w_3 c_3^3 &= 1/4, \\ w_1 c_1^4 + w_2 c_2^4 + w_3 c_3^4 &= 1/5, \\ w_1 c_1^5 + w_2 c_2^5 + w_3 c_3^5 &= 1/6, \end{aligned}$$

These conditions ensure that over the interval [0; 1] the rule integrates exactly all polynomials of degree at most 5. (a) Numerically solve the system using Newton's method. (b) Using your results from (a), write a function to implement a composite Gauss-Legendre rule to approximate the integral

$$\int_{-\pi/2}^{\pi/2} e^x cos(5x)dx$$

using 2, 4, 8, 16, and 32 subintervals. Compare with the exact answer, and plot the partition number versus the error.

Solution.

(a) We seek a three-point quadrature formula on $[0, 1]$:

$$\int_0^1 f(t)\,dt \approx \sum_{i=1}^{3} w_i f(c_i).$$

To ensure exactness for polynomials of degree at most 5, we impose the moment conditions

$$\sum_{i=1}^{3} w_i c_i^k = \frac{1}{k+1}, \qquad k = 0, \ldots, 5.$$

This gives a nonlinear system of six equations in the six unknowns $w_1, w_2, w_3, c_1, c_2, c_3$, which we solve using Newton's method:

$$J(z)\Delta z = -F(z), \qquad z_{\text{new}} = z + \Delta z.$$

```
1
2  import numpy as np
3
4  def find_nodes_weights(tol=1e-13, max_iter=100):
5      z = np.array([1/3, 1/3, 1/3, 0.1, 0.5, 0.9], dtype=float)
6
7      for _ in range(max_iter):
8          w, c = z[:3], z[3:]
9          F = np.array([np.sum(w * c**k) - 1/(k+1) for k in range(6)])
10
11         J = np.zeros((6, 6))
12         for k in range(6):
13             J[k, :3] = c**k
14             if k:
15                 J[k, 3:] = k * w * c**(k-1)
16
17         dz = np.linalg.solve(J, -F)
18         z += dz
19         if np.linalg.norm(dz) < tol:
```

```
            return z[:3], z[3:]

    raise RuntimeError("Newton's method did not converge")

w, c = find_nodes_weights()
print("Weights:", w)
print("Nodes:", c)

output:

Weights: [0.27777778 0.44444444 0.27777778]
Nodes: [0.11270167 0.5        0.88729833]
```

(b) We approximate

$$I = \int_{-\pi/2}^{\pi/2} e^x \cos(5x)\, dx.$$

Divide $[a, b] = [-\pi/2, \pi/2]$ into N equal subintervals of length $h = (b - a)/N$. Using the nodes and weights from part (a), the composite quadrature is

$$I_N = h \sum_{k=0}^{N-1} \sum_{i=1}^{3} w_i f(a + kh + c_i h).$$

The exact integral is

$$I = \frac{5}{26}\left(e^{\pi/2} + e^{-\pi/2}\right),$$

so the absolute error is $E_N = |I - I_N|$.

```

import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.exp(x) * np.cos(5*x)

def composite_gauss(N, w, c):
    a, b = -np.pi/2, np.pi/2
    h = (b - a) / N
    x = a + h * (np.arange(N)[:, None] + c)
    return h * np.sum(f(x) * w)

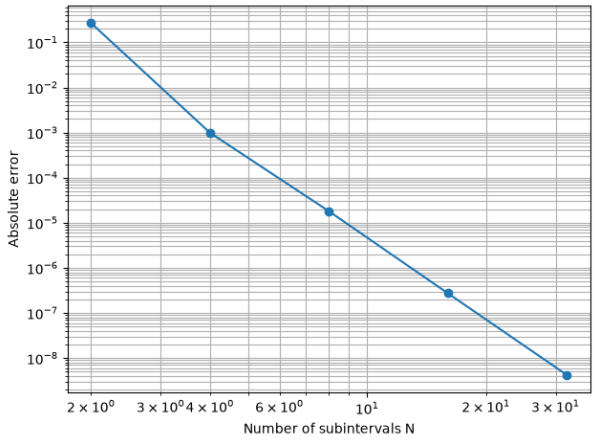
exact = 5/26 * (np.exp(np.pi/2) + np.exp(-np.pi/2))
Ns = np.array([2, 4, 8, 16, 32])
errors = []

for N in Ns:
    approx = composite_gauss(N, w, c)
    error = abs(exact - approx)
    errors.append(error)
    print(f"N={N:2d}, approximation={approx:.12f}, error={error:.6e}")

plt.loglog(Ns, errors, "o-")
plt.xlabel("Number of subintervals N")
plt.ylabel("Absolute error")
plt.grid(True, which="both")
plt.tight_layout()
plt.show()


output:
N= 2, approximation=0.698271333330, error=2.667973e-01
```

```
N= 4, approximation=0.966057559379, error=9.889137e-04
N= 8, approximation=0.965086670432, error=1.802479e-05
N=16, approximation=0.965068919711, error=2.740728e-07
N=32, approximation=0.965068649885, error=4.246930e-09
```



Problem 11. In class we discussed both the error estimate

$$|\int_a^b f(x)dx - Q_{NC(3)}| = \frac{1}{2880}M_4(b-a)^5,$$

for the three-point, closed, Newton-Cotes (Simpson) rule, as well as the error estimate

$$|\int_a^b f(x)dx - Q_{NC(3)}| = \frac{1}{6480}M_4(b-a)^5,$$

for the four-point, closed, Newton-Cotes (Simpson 3/8) rule, where in each case M4 is a bound on $|f(4)(x)|$ for $x \in [a, b]$. As remarked, the Simpson 3/8 rule has a slightly better error bound, but requires one extra function evaluation. The similar scaling in these estimates suggests that the Simpson rule is superior to the Simpson 3/8 rule if our measure is accuracy per function evaluation.
That the Simpson rule is indeed better in this measure may be confirmed in the composite-rule setting. For example, both $Q^(3)_{NC(3)}$ and $Q^(2)_{NC(4)}$ are based on $3(31) + 1 = 2(41) + 1 = 7$ points, whence each requires seven function evaluations.
(a) Approximate $\int_0^2 arctan(x)dx$ with each of the seven-point composite rules above, computing errors against the exact answer obtained by the Fundamental Theorem of Calculus. (b) Use the general error formula to prove that, in the composite-rule setting, the error estimate for the Simpson rule is better (in terms of accuracy per function evaluation) than the one for the Simpson 3/8 rule.

Solution.

(a) The exact integral is

$$I = \int_0^2 \arctan(x)\,dx = \Big[x\arctan(x) - \frac{1}{2}\ln(1+x^2)\Big]_0^2 = 2\arctan(2) - \frac{1}{2}\ln 5.$$

Both composite rules use seven function evaluations. The composite Simpson rule uses $n = 3$ subintervals, while the composite Simpson 3/8 rule uses $n = 2$ subintervals.
For $f(x) = \arctan(x)$, we compute both approximations and their absolute errors:

$$E_S = |I - Q_S|, \qquad E_{3/8} = |I - Q_{3/8}|.$$

```

import numpy as np

f = np.arctan
a, b = 0.0, 2.0
I = 2*np.arctan(2) - 0.5*np.log(5)
```

```
7
8  xS = np.linspace(a, b, 7)
9  hS = (b-a)/6
10 QS = hS/3 * (f(xS[0]) + f(xS[-1])
11               + 4*np.sum(f(xS[1:-1:2]))
12               + 2*np.sum(f(xS[2:-1:2])))
13
14
15 x38 = np.linspace(a, b, 7)
16 h38 = (b-a)/6
17 Q38 = 3*h38/8 * (
18     f(x38[0]) + 3*f(x38[1]) + 3*f(x38[2]) + f(x38[3])
19     + f(x38[3]) + 3*f(x38[4]) + 3*f(x38[5]) + f(x38[6])
20 )
21
22 print(f"Exact integral: {I:.10f}")
23 print(f"Composite Simpson: {QS:.10f}, error = {abs(I-QS):.6e}")
24 print(f"Composite Simpson 3/8: {Q38:.10f}, error = {abs(I-Q38):.6e}")
25
26
27 output:
28 Exact integral: 1.4095784794
29 Composite Simpson: 1.4097607265, error = 1.822471e-04
30 Composite Simpson 3/8: 1.4100275815, error = 4.491021e-04
```

(b) Let $L = b - a$ and $M_4 = \max_{[a,b]} |f^{(4)}|$. Split $[a, b]$ into N panels of width L/N, and let K be the total number of function evaluations. The single-panel bounds apply to each panel, so we sum over N panels.

Simpson: $K = 2N + 1$, so $N = \frac{K-1}{2}$, and

$$E_S \le N \cdot \frac{M_4}{2880}\left(\frac{L}{N}\right)^5 = \frac{M_4 L^5}{2880\, N^4} = \frac{16\, M_4 L^5}{2880\,(K-1)^4} = \frac{M_4 L^5}{180\,(K-1)^4}.$$

Simpson 3/8: $K = 3N + 1$, so $N = \frac{K-1}{3}$, and

$$E_{3/8} \le N \cdot \frac{M_4}{6480}\left(\frac{L}{N}\right)^5 = \frac{M_4 L^5}{6480\, N^4} = \frac{81\, M_4 L^5}{6480\,(K-1)^4} = \frac{M_4 L^5}{80\,(K-1)^4}.$$

Since $\frac{1}{180} < \frac{1}{80}$, for the same number K of function evaluations the Simpson bound is smaller by a factor $\frac{180^{-1}}{80^{-1}} = \frac{4}{9}$ (i.e. the 3/8 bound is 2.25 times larger). Hence Simpson's rule is better per function evaluation. This agrees with part (a), where $E_S \approx 1.8 \cdot 10^{-4}$ and $E_{3/8} \approx 4.5 \cdot 10^{-4}$ for $K = 7$.